

Lab4 实验报告

一、思考题

1、系统调用的实现

- 内核在保存现场的时候是如何避免破坏通用寄存器的？

系统从用户态切换到内核态后，内核首先需要将原用户进程的运行现场保存到内核空间，这一过程在 kern/entry.S 中通过 SAVE_ALL 宏完成。观察 SAVE_ALL 宏可发现此时已经将寄存器中的值全部存入 Trapframe 结构体的对应位置。

```
.macro SAVE_ALL
.set noat
.set noreorder
    mfc0    k0, CP0_STATUS
    andi    k0, STATUS_UM
    beqz    k0, 1f
    move    k0, sp
    /*
    * If STATUS_UM is not set, the exception was triggered in kernel mode.
    * $sp is already a kernel stack pointer, we don't need to set it again.
    */
    li      sp, KSTACKTOP
1:
    subu    sp, sp, TF_SIZE
    sw      k0, TF_REG29(sp)
    mfc0    k0, CP0_STATUS
    sw      k0, TF_STATUS(sp)
    mfc0    k0, CP0_CAUSE
    sw      k0, TF_CAUSE(sp)
    mfc0    k0, CP0_EPC
    sw      k0, TF_EPC(sp)
    mfc0    k0, CP0_BADVADDR
    sw      k0, TF_BADVADDR(sp)
    mfhi    k0
    sw      k0, TF_HI(sp)
    mflo    k0
    sw      k0, TF_LO(sp)
    sw      $0, TF_REG0(sp)
    sw      $1, TF_REG1(sp)
    sw      $2, TF_REG2(sp)
    sw      $3, TF_REG3(sp)
    sw      $4, TF_REG4(sp)
    sw      $5, TF_REG5(sp)
    sw      $6, TF_REG6(sp)
    sw      $7, TF_REG7(sp)
    sw      $8, TF_REG8(sp)
    sw      $9, TF_REG9(sp)
    sw      $10, TF_REG10(sp)
    sw      $11, TF_REG11(sp)
    sw      $12, TF_REG12(sp)
    sw      $13, TF_REG13(sp)
```

```

    SW      $14, TF_REG14(sp)
    SW      $15, TF_REG15(sp)
    SW      $16, TF_REG16(sp)
    SW      $17, TF_REG17(sp)
    SW      $18, TF_REG18(sp)
    SW      $19, TF_REG19(sp)
    SW      $20, TF_REG20(sp)
    SW      $21, TF_REG21(sp)
    SW      $22, TF_REG22(sp)
    SW      $23, TF_REG23(sp)
    SW      $24, TF_REG24(sp)
    SW      $25, TF_REG25(sp)
    SW      $26, TF_REG26(sp)
    SW      $27, TF_REG27(sp)
    SW      $28, TF_REG28(sp)
    SW      $30, TF_REG30(sp)
    SW      $31, TF_REG31(sp)
.set at
.set reorder
.endm

```

- 系统陷入内核调用后可以直接从当时的\$a0-\$a3参数寄存器中得到用户调用msyscall留下的信息吗？

可以，\$a0-\$a3参数寄存器未被修改过。

- 我们是怎么做到让sys开头的函数“认为”我们提供了和用户调用msyscall时同样的参数的？

在用户程序调用msyscall时，我们将参数按约定放入寄存器和栈中。

- 内核处理系统调用的过程对Trapframe做了哪些更改？这种修改对应的用户态的变化是什么？

将Trapframe中的cp0_epc设置为系统调用返回后的下一条指令地址，将用户态的通用寄存器保存到Trapframe中。这种修改保护PC被恢复为系统调用后的下一条指令，用户态程序继续正常执行。并且系统调用返回时，Trapframe中保存的寄存器值会被恢复

2、envid2env 的实现

- 思考 envid2env函数: 为什么 envid2env中需要判断 e->env_id != envid的情况？如果没有这步判断会发生什么情况？

当进程被销毁并重新分配时，env_id 的版本号会递增，确保新进程的 env_id 不会与旧进程混淆。如果没有这步判断可能因为复用导致混淆问题或返回错误的id导致越权访问。

3、mkenvid 函数细节

- 思考下面的问题，并对这个问题谈谈你的理解：请回顾 kern/env.c 文件中 mkenvid() 函数的实现，该函数不会返回 0，请结合系统调用和 IPC 部分的实现与envid2env() 函数的行为进行解释。

envid = 0 被保留为特殊值，表示“当前进程”，例如在 envid2env() 中直接返回 curenv，因此 mkenvid() 不返回 0 可避免二义性。可以说envid 的严格管理是进程隔离和通信安全的基础。

4、fork 的返回结果

- 关于 fork 函数的两个返回值，下面说法正确的是：
 - A、fork 在父进程中被调用两次，产生两个返回值
 - B、fork 在两个进程中分别被调用一次，产生两个不同的返回值
 - C、fork 只在父进程中被调用了一次，在两个进程中各产生一个返回值
 - D、fork 只在子进程中被调用了一次，在两个进程中各产生一个返回值

C

1、fork 之前只有父进程存在。2、fork 之后，父子进程同时开始执行fork之后的代码段。3、fork 在不同的进程中返回值不一样，在子进程中返回值为0，在父进程中返回值不为0，而为子进程的 pid（Linux 中进程专属的id，类似于MOS中的envid）。

5、用户空间的保护

- 我们并不应该对所有的用户空间页都使用duppage进行映射。那么究竟哪些用户空间页应该映射，哪些不应该呢？请结合kern/env.c中env_init函数进行的页面映射、include/mmu.h里的内存布局图以及本章的后续描述进行思考。

需要保护从 UTEXT 到 USTACKTOP 的用户空间页。其余部分要么是被所有进程共用，要么是异常栈用于处理异常的空间。

6、vpt 的使用

在遍历地址空间存取页表项时你需要使用到vpd和vpt这两个指针，请参考user/include/lib.h中的相关定义，思考并回答这几个问题：

- vpt和vpd的作用是什么？怎样使用它们？

vpt（Virtual Page Table）指向当前进程的页表项数组，即页表基地址，可通过 vpt[VPN(va)] 直接访问虚拟地址 va 对应的 PTE。vpd（Virtual Page Directory）指向当前进程的页目录项数组，即页目录基地址，可通过 vpd[PDX(va)] 访问虚拟地址 va 对应的 PDE。通过vpd和vpt可以访问到页表项数组中的页表项，从而实现虚拟地址到物理地址的映射。

- 从实现的角度谈一下为什么进程能够通过这种方式来存取自身的页表？

这是通过页表自映射实现的。

- 它们是如何体现自映射设计的？

`e->env_pgdir[PDX(UVPT)] = PADDR(e->env_pgdir) | PTE_V`，`PADDR(e->env_pgdir)`是把当前进程页目录的内核虚拟地址转化为物理地址，并且将有效位置一。而 `PDX(UVPT)` 是获取 UVPT（User Virtual Page Table）的页目录索引，这样就将用户环境的页目录地址写入了页目录中索引为 `PDX(UVPT)` 的位置，从而实现了页目录自映射，使得用户空间可以通过 UVPT 访问自己的页表结构。

- 进程能够通过这种方式来修改自己的页表项吗

不能，用户进程页表项没有写的权限，无法修改自己和内核的页表项。

7、页写入异常-内核处理

在 `do_tlb_mod` 函数中，你可能注意到了一个向异常处理栈复制 `Trapframe` 运行现场的过程，请思考并回答这几个问题：

- 这里实现了一个支持类似于“异常重入”的机制，而在什么时候会出现这种“异常重入”？

内核在处理 写时复制（COW）时，可能需要分配新物理页或修改页表，而这些操作本身可能再次触发缺页异常。

- 内核为什么需要将异常的现场 `Trapframe` 复制到用户空间？

缺页中断的处理由用户进程负责，所以用户进程需要读取 `Trapframe` 的值获得触发缺页的指令和缺的页来进行调页。并且用户进程处理完毕恢复现场的时候也要使用 `Trapframe` 的数据。

8、页写入异常-用户处理-1

- 在用户态处理页写入异常，相比于在内核态处理有什么优势？

1) 如果每次异常都需要进入内核，保存并恢复完整的上下文，开销较大。2) 若内核直接修改用户数据，可能被恶意程序利用（如通过缺页异常触发内核漏洞）。

9、页写入异常-用户处理-2

请思考并回答以下几个问题：

- 为什么需要将 `syscall_set_tlb_mod_entry` 的调用放置在 `syscall_exofork` 之前？

1、确保子进程继承异常处理机制；2、避免异常处理未定义。

- 如果放置在写时复制保护机制完成之后会有怎样的效果？

在没有定义异常处理函数的情况下，页写入异常将无法被捕获和处理，可能导致程序崩溃或行为异常。

二、难点分析

本次实验第一个难点我认为是完成函数 `envid2env`。虽然这个函数逻辑简单，但是需要仔细依照给的提示考虑各种情况，需考虑好返回 `-E_BAD_ENV` 的各种情况。

虽然这次实验的大部分练习都在 `kern/syscall_all.c` 中，但是大部分函数的实现都很简单，每一步基本上只有一个语句，逻辑清晰没有歧义。比较复杂的函数实现有 `sys_set_env_status`，需要在 `env` 的状态改变后更新 `env_sched_list`。注意 `env` 现在的状态必须与原状态不同才能更新调度队列。此外练习8也比较复杂，需要完成 `ipc` 通信两端信息的发送与接收，步骤较多，稍有不慎就会犯错。

`do_syscall` 函数的实现对我也有点难度，因为我对栈和寄存器不太熟，一开始不知道该怎么保存现场，将值存到 `Trapframe` 结构体中。

`use/lib/fork.c` 的 `duppage` 和 `fork` 两个函数实现起来也比较复杂，前者需要考虑页表项原来的权限位来赋给他新的值，并给出正确的异常处理。后者需注意一级页表有效时才能进入二级页表使用 `duppage` 给页表项添加权限位。

三、实验体会

这次实验的主要内容是掌握系统调用的概念与流程，实现IPC、fork函数和页写入异常的处理。不得不说，几次线下实验的内容越来越多，涵盖的东西也更复杂。本来前几次实验就只是掌握得“知其然不知其所以然”，有些内容理解起来吃力，都是打个马虎就过去了。这次实验内容更多，整体也是跟着指导书得练习题来学习，一些练习题没有涉及到的内容也理解得不是很透彻。而且这篇实验报告写于五一假期最后一天，距我完成Lab4课下实验也过去了一周，本来很多内容就一知半解，这下经过一周的时间，本来掌握的一点点知识更是蒙上了一层面纱，这个实验报告也算是帮我回忆了一些遗忘的知识，希望周四的课上实验能取得好的成绩。

四、原创说明

- 本文部分内容在LLM（大语言模型）的协助下完成